Republic of the Philippines

University of Southern Mindanao

College of Engineering and Information Technology

DEPARTMENT OF COMPUTING AND INFORMATION SCIENCE

HOMEWORK: K-Nearest Neighbors (KNN) on Diabetes Dataset

Course: Computational Science for Computer Science

Topic: Machine Learning – K-Nearest Neighbors

Objective

This homework aims to develop your understanding of instance-based learning by applying the K-Nearest Neighbors (KNN) algorithm to a real-world dataset. You will analyze, preprocess, and classify data to predict diabetes outcomes.

Dataset Description

You are provided with a dataset containing 768 patient records with the following features:

• Pregnancies

• Glucose

• BloodPressure

• SkinThickness

• Insulin

• BMI

• DiabetesPedigreeFunction

• Age

• Outcome (Target Variable: 0 = Nondiabetic, 1 = Diabetic)

Dataset:

diabetes_knn.csv

https://drive.google.com/drive/u/2/folders/1UA5YAHxIre0XG1YCEICbJ_x_uGji5inu

768 total records – 8 features – 500 non-diabetic (0) – 268 diabetic (1)

Part 1: Data Understanding (10 pts)

1. Describe each feature in your own words.

2. Identify:

- Which features are likely most important for prediction.
    a. **The most important features for prediction are Glucose, BMI, and Age.**

    Glucose is the most directly linked to diabetes by definition. Diabetes *is* a disorder of blood glucose regulation — the body either doesn't produce enough insulin (Type 1) or doesn't respond to it properly (Type 2), causing glucose to stay elevated.

    BMI matters because excess body fat especially visceral/abdominal fat causes **insulin resistance** — the cells stop responding to insulin signals, forcing the pancreas to overproduce, eventually leading to Type 2 diabetes. Which features may contain missing or problematic values.

    Age matters because Type 2 diabetes is a progressive condition. The longer you live, the more years of wear on your pancreatic beta cells (which produce insulin), the more time for insulin resistance to accumulate, and the less efficiently your body handles glucose.

    b. **The most problematic are Insulin (374 zeros) and SkinThickness (227 zeros).**

    Insulin — everyone has some insulin circulating in their blood. A reading of 0 mU/L would mean the patient has absolutely no insulin, which would be an immediately life-threatening emergency (diabetic ketoacidosis). You'd be in an ICU, not filling out a questionnaire. So, a zero doesn't mean "zero insulin" — it almost certainly means the measurement was not taken and someone filled in 0 as a placeholder.

    SkinThickness — similarly, everyone has measurable skin and subcutaneous fat. A skinfold thickness of 0 mm is physically impossible on a living human. Again, 0 here means **data not collected**, not an actual reading.

    The reason this is dangerous for kNN specifically more than for some other algorithms is that kNN uses **raw values to compute distance**. If you leave those zeros in, a patient with a recorded insulin of 0 looks extremely closely in distance to another patient with insulin of 0 — but they might both just have missing data, not actually similar insulin profiles.

Part 2: Data Preprocessing (20 pts)

1. Check for missing or zero values (e.g., Insulin, SkinThickness).

2. Handle missing data using one of the following:

Median Imputation

I. **Scan each column** — find all zeros (impossible values). 768 total rows.

Feature	Zero Count
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11

II. **Compute the median** — using ONLY the valid (non-zero) values

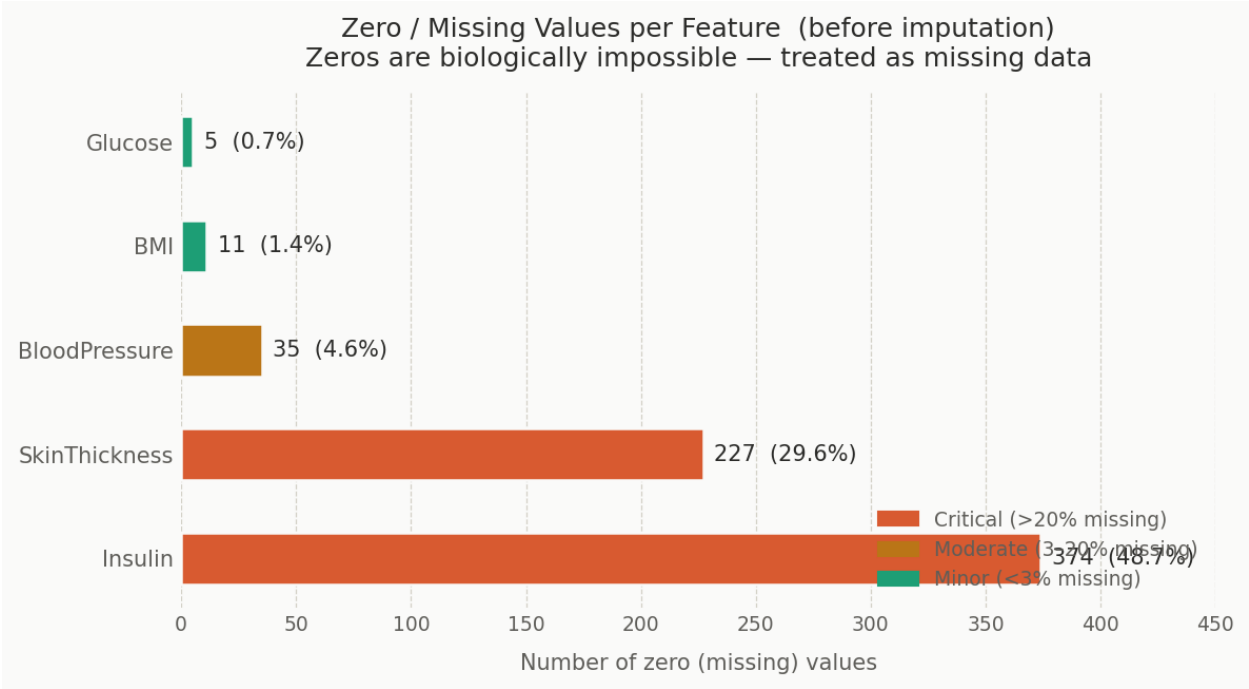
median = middle value when all valid values are sorted from smallest to largest

Feature	Valid (Non-Zero) Rows	Min	Max	Median Computed
Glucose	763 values	44	199	117
BloodPressure	733 values	24	122	72
SkinThickness	541 values	7	99	29
Insulin	394 values	14	846	125
BMI	757 values	18.2	67.1	32.3

III. Replace every zero with its column's median — row by row. View file and/or table in this link below:

diabetes_imputed.csv

https://drive.google.com/drive/u/2/folders/1UA5YAHxIre0XG1YCEICbJ_x_uGji5inu



3. Apply feature scaling (required for KNN):

Standardization

i. Formula:
$z = (x - \mu) \div \sigma$

Symbol	Meaning	Interpretation
x	Original value	The raw number from your dataset
μ (mu)	Column mean	Average of ALL values in that column
σ (sigma)	Column std dev	How spread out the values are
z	Scaled value	The final number

MEAN (μ) — the average

Formula: $\mu = \mathbf{\Sigma x \div N}$ Σx = sum of ALL values in the column
N = total number of rows (768)

STD DEV (σ) — how spread-out values are

Formula: $\sigma = \sqrt{(\mathbf{\Sigma(x-\mu)^2 \div (N-1)})}$ $\Sigma(x-\mu)^2$ = sum of squared differences from mean
N−1 = 767 (sample standard deviation)

The result z tells you: is this value above average (+) or below average (−), and by how much? A z of +1.0 means "1 standard deviation above the mean". A z of −2.0 means "2 standard deviations below the mean". Every column ends up with mean=0 and std=1.

ii. **Compute μ and σ for each column**

Feature	**Mean (μ)**	**Std Dev (σ)**	Interpretation
Pregnancies	**3.845**	**3.370**	Average patient had -4 pregnancies
Glucose	**121.656**	**30.438**	Average blood glucose -122 mg/dL
BloodPressure	**72.387**	**12.097**	Average BP -72 mmHg
SkinThickness	**29.108**	**8.791**	Average skinfold -29 mm
Insulin	**140.672**	**86.383**	Average insulin -141 mU/L
BMI	**32.455**	**6.875**	Average BMI -32.5
DiabetesPedigreeFunction	**0.472**	**0.331**	Average DPF -0.47
Age	**33.241**	**11.760**	Average age -33 years

iii. **Apply the formula — Row 0, every feature**

Pick a feature to see the full step-by-step breakdown for Row 0 (Patient: 6 pregnancies, Glucose=148, BP=72, Skin=35, Insulin=125, BMI=33.6, DPF=0.627, Age=50):

Pregnancies:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (\mathbf{6} - \mathbf{3.8451}) \div \mathbf{3.3696}$

STEP 1 — subtract mean: 6 − 3.8451 = **2.154948**

STEP 2 — divide by std: 2.154948 ÷ 3.3696 = **0.639531**

VALUE: **0.6399**

Glucose:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (\mathbf{148} - \mathbf{121.6563}) \div \mathbf{30.4383}$

STEP 1 — subtract mean: 148 − 121.6563 = **26.343750**

STEP 2 — divide by std: 26.343750 ÷ 30.4383 = **0.865481**

VALUE: **0.8660**

BloodPressure:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (72 - 72.3867) \div 12.0966$

STEP 1 — subtract mean: 72 − 72.3867 = -0.386719

STEP 2 — divide by std: -0.386719 ÷ 12.0966 = -0.031969

VALUE: -0.0320

SkinThickness:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (35 - 29.1081) \div 8.7912$

STEP 1 — subtract mean: 35 − 29.1081 = 5.891927

STEP 2 — divide by std: 5.891927 ÷ 8.7912 = 0.670206

VALUE: **0.6706**

Insulin:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (125 - 140.6719) \div 86.3831$

STEP 1 — subtract mean: 125 − 140.6719 = -15.671875

STEP 2 — divide by std: -15.671875 ÷ 86.3831 = -0.181423

VALUE: **-0.1815**

BMI:

FORMULA: $z = (x - \mu) \div \sigma$

PLUG IN: $z = (33.6 - 32.4552) \div 6.8752$

STEP 1 — subtract mean: 33.6 − 32.4552 = 1.144792

STEP 2 — divide by std: 1.144792 ÷ 6.8752 = 0.166511

VALUE**: 0.1666**

DPF:

FORMULA: z = (x − μ) ÷ σ

PLUG IN: z = (0.627 − 0.4719) ÷ 0.3313

STEP 1 — subtract mean: 0.627 − 0.4719 = 0.155124

STEP 2 — divide by std: 0.155124 ÷ 0.3313 = 0.468187

VALUE: **0.4685**

Age:

FORMULA: z = (x − μ) ÷ σ

PLUG IN: z = (50 − 33.2409) ÷ 11.7602

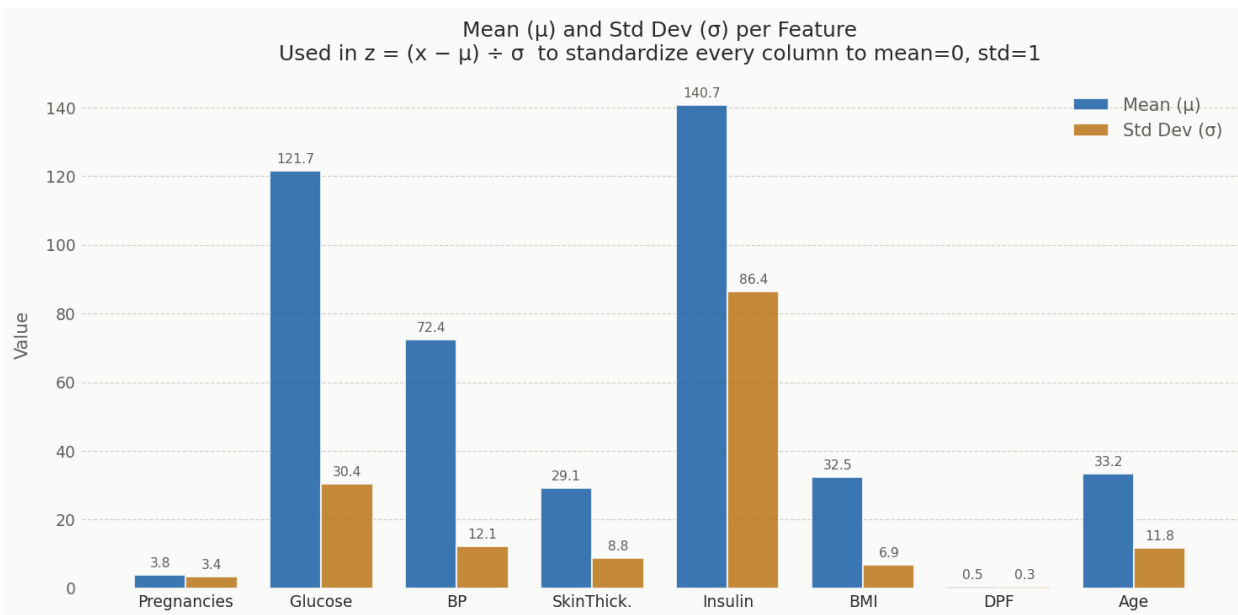
STEP 1 — subtract mean: 50 − 33.2409 = 16.759115

STEP 2 — divide by std: 16.759115 ÷ 11.7602 = 1.425067

VALUE: **1.4260**

Continue this to all the rows on the data set. To view the file to navigate all rows for the scaled diabetes dataset, click this link below:

diabetes_scaled.csv
https://drive.google.com/drive/u/2/folders/1UA5YAHxIre0XG1YCEICbJ_x_uGji5inu



4. Show before and after preprocessing results (tables or summary statistics).

Sample Rows Before & After Comparison (First 5 patients)

Stage 1: Raw Data

Row	Preg.	Glucose	BloodPres.	SkinThickn	Insulin	BMI	DPF	Age	Outcome
0	6	148	72	35	**0**	33.6	0.627	50	1
1	1	85	66	29	**0**	26.6	0.351	31	0
2	8	183	64	**0**	**0**	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Stage 2: After Imputation

Row	Preg.	Glucose	BloodPres.	SkinThick.	Insulin	BMI	DPF	Age	Outcome
0	6	148	72	35	**125**	33.6	0.627	50	1
1	1	85	66	29	**125**	26.6	0.351	31	0
2	8	183	64	**29**	**125**	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Stage 3: After Scaling

Row	Preg.	Glucose	BloodPres.	SkinThick.	Insulin	BMI	DPF	Age	Outcome
0	0.6399	0.866	-0.032	0.6706	-0.1815	0.1666	0.4685	1.426	1
1	-0.8449	-1.2051	-0.5283	-0.0123	-0.1815	-0.8522	-0.3651	-0.1907	0
2	1.2339	2.0167	-0.6938	-0.0123	-0.1815	-1.3325	0.6044	-0.1056	1

Row	Preg.	Glucose	BloodPres.	SkinThick.	Insulin	BMI	DPF	Age	Outcome
3	-0.8449	-1.0736	-0.5283	-0.6952	-0.5406	-0.6339	-0.9208	-1.0415	0
4	-1.1419	0.5044	-2.6791	0.6706	0.3166	1.5493	5.4849	-0.0205	1

Zero Counts — Before vs After Imputation – Summary Statistics

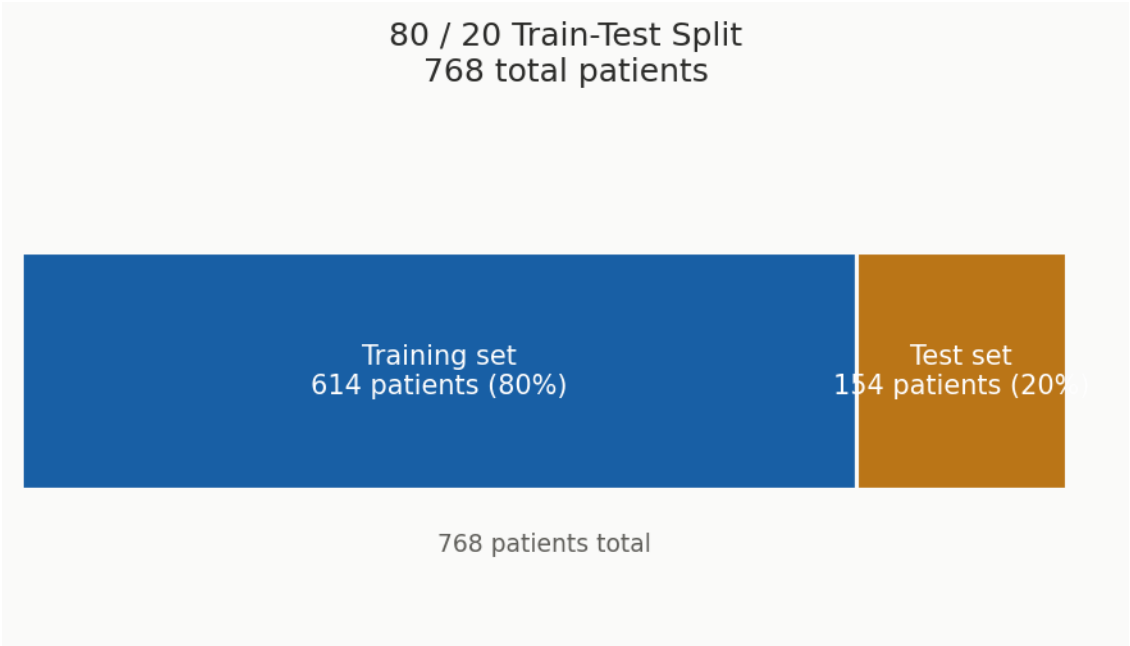
Feature	Before Zeros	% Missing	Median Applied	After Zeros
Insulin	374	48.7%	125.0 mU/L	0
SkinThickness	227	29.6%	29.0 mm	0
BloodPressure	35	4.6%	72.0 mmHg	0
BMI	11	1.4%	32.3 kg/m^2	0
Glucose	5	0.7%	117.0 mg/dL	0

Part 3: KNN Implementation (30 pts)

1. Split the dataset:

- 80% Training
- 20% Testing

Set	**Rows**	**% of Total**
Training	614	80%
Testing	154	20%
Total	768	100%



2. Train a KNN model using at least three different K values:

- Example: K = 3, 5, 7

3. Use Euclidean Distance:

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

4. For ONE selected test instance:

Selected Test Instance (Patient #669 from diabetes-dataset)

Feature	**Raw Value**	**Scaled Value**
Pregnancies	6	**0.6399**
Glucose	98 mg/dL	**−0.7777**
BloodPressure	58 mmHg	**−1.1901**
SkinThickness	33 mm	**0.4430**
Insulin	190 mU/L	**0.5714**

Feature	Raw Value	Scaled Value
BMI	34.0 kg/m^2	**0.2248**
DiabetesPedigreeFunction	0.430	**−0.1265**
Age	43 years	**0.8304**

Actual outcome = Non-diabetic (0). We will use this to check if kNN predicts correctly.

- Manually compute distances to at least 10 training samples
- Show the step-by-step computation
- Identify nearest neighbors and predicted class

Formula: **$d = \sqrt{[\ \Sigma(test_i - train_i)^2\ ]}$**

Train #0 (CSV row #16) — Actual: Diabetic (1)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	7	+0.6399	+0.9369	−0.2970	0.0882
Glucose	100	−0.7777	−0.7119	−0.0658	0.0043
BloodPressure	0→72*	−1.1901	−0.0320	−1.1581	1.3412
SkinThickness	0→29*	+0.4430	−0.0123	+0.4553	0.2073
Insulin	0→125*	+0.5714	−0.1815	+0.7529	0.5669
BMI	30.0	+0.2248	−0.3573	+0.5821	0.3389
DPF	0.484	−0.1265	+0.0366	−0.1631	0.0266
Age	32	+0.8304	−0.1056	+0.9360	0.8761

Sum of $(x-y)^2$ = 0.0882 + 0.0043 + 1.3412 + 0.2073 + 0.5669 + 0.3389 + 0.0266 + 0.8761 = **3.4495**
Distance = $\sqrt{3.4495}$ = 1.8573

Train #1 (CSV row #43) — Actual: Non-diabetic (0)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	7	+0.6399	+0.9369	−0.2970	0.0882

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Glucose	106	−0.7777	−0.5147	−0.2630	0.0692
BloodPressure	92	−1.1901	+1.6224	−2.8125	7.9104
SkinThickness	18	+0.4430	−1.2644	+1.7074	2.9151
Insulin	0→125*	+0.5714	−0.1815	+0.7529	0.5669
BMI	22.7	+0.2248	−1.4198	+1.6446	2.7048
DPF	0.235	−0.1265	−0.7154	+0.5889	0.3468
Age	48	+0.8304	+1.2558	−0.4254	0.1810

Sum of $(x-y)^2$ = 0.0882 + 0.0692 + 7.9104 + 2.9151 + 0.5669 + 2.7048 + 0.3468 + 0.1810 = **14.7824**
Distance = √14.7824 = 3.8448

Train #2 (CSV row #88) — Actual: Non-diabetic (0)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	2	+0.6399	−0.5479	+1.1878	1.4109
Glucose	100	−0.7777	−0.7119	−0.0658	0.0043
BloodPressure	68	−1.1901	−0.3629	−0.8272	0.6843
SkinThickness	25	+0.4430	−0.4676	+0.9106	0.8292
Insulin	71	+0.5714	−0.8071	+1.3785	1.9002
BMI	38.5	+0.2248	+0.8798	−0.6550	0.4290
DPF	0.324	−0.1265	−0.4466	+0.3201	0.1025
Age	26	+0.8304	−0.6161	+1.4465	2.0924

Sum of $(x-y)^2$ = 1.4109 + 0.0043 + 0.6843 + 0.8292 + 1.9002 + 0.4290 + 0.1025 + 2.0924 = **7.4528**
Distance = √7.4528 = 2.7300

Train #3 (CSV row #131) — Actual: Diabetic (1)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	4	+0.6399	+0.0460	+0.5939	0.3527
Glucose	173	−0.7777	+1.6879	−2.4656	6.0793
BloodPressure	70	−1.1901	−0.1974	−0.9927	0.9854
SkinThickness	14	+0.4430	−1.7197	+2.1627	4.6771
Insulin	168	+0.5714	+0.3166	+0.2548	0.0649
BMI	29.7	+0.2248	−0.4010	+0.6258	0.3916
DPF	0.361	−0.1265	−0.3349	+0.2084	0.0434
Age	33	+0.8304	−0.0205	+0.8509	0.7240

Sum of $(x-y)^2$ = 0.3527 + 6.0793 + 0.9854 + 4.6771 + 0.0649 + 0.3916 + 0.0434 + 0.7240 = **13.3185**
Distance = √13.3185 = 3.6494

Train #4 (CSV row #176) — Actual: Diabetic (1)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	8	+0.6399	+1.2339	−0.5940	0.3528
Glucose	179	−0.7777	+1.8852	−2.6629	7.0908
BloodPressure	72	−1.1901	−0.0320	−1.1581	1.3412
SkinThickness	42	+0.4430	+1.4674	−1.0244	1.0494
Insulin	130	+0.5714	−0.1236	+0.6950	0.4831
BMI	32.7	+0.2248	+0.0356	+0.1892	0.0358
DPF	0.719	−0.1265	+0.7463	−0.8728	0.7619
Age	36	+0.8304	+0.2348	+0.5956	0.3548

Sum of $(x-y)^2$ = 0.3528 + 7.0908 + 1.3412 + 1.0494 + 0.4831 + 0.0358 + 0.7619 + 0.3548 = **11.4698**
Distance = √11.4698 = 3.3867

Train #5 (CSV row #221) — Actual: Diabetic (1)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	0	+0.6399	−1.1419	+1.7818	3.1746
Glucose	177	−0.7777	+1.8194	−2.5971	6.7450
BloodPressure	60	−1.1901	−1.0246	−0.1655	0.0274
SkinThickness	0→29*	+0.4430	−0.0123	+0.4553	0.2073
Insulin	478	+0.5714	+3.9076	−3.3362	11.1300
BMI	34.6	+0.2248	+0.3122	−0.0874	0.0076
DPF	1.072	−0.1265	+1.8124	−1.9389	3.7595
Age	21	+0.8304	−1.0415	+1.8719	3.5042

Sum of $(x-y)^2$ = 3.1746 + 6.7450 + 0.0274 + 0.2073 + 11.1300 + 0.0076 + 3.7595 + 3.5042 = **28.5557**
Distance = √28.5557 = 5.3438

Train #6 (CSV row #311) — Actual: Non-diabetic (0)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	6	+0.6399	+0.6399	0.0000	0.0000
Glucose	80	−0.7777	−1.3694	+0.5917	0.3502
BloodPressure	66	−1.1901	−0.5283	−0.6618	0.4380
SkinThickness	30	+0.4430	+0.1015	+0.3415	0.1166
Insulin	0→125*	+0.5714	−0.1815	+0.7529	0.5669
BMI	26.2	+0.2248	−0.9104	+1.1352	1.2887
DPF	0.313	−0.1265	−0.4798	+0.3533	0.1248
Age	41	+0.8304	+0.6602	+0.1702	0.0290

Sum of $(x-y)^2$ = 0.0000 + 0.3502 + 0.4380 + 0.1166 + 0.5669 + 1.2887 + 0.1248 + 0.0290 = **2.9142**
Distance = √2.9142 = 1.7071

Train #7 (CSV row #376) — Actual: Diabetic (1)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	12	+0.6399	+2.4217	−1.7818	3.1750
Glucose	140	−0.7777	+0.6030	−1.3807	1.9065
BloodPressure	82	−1.1901	+0.7952	−1.9853	3.9415
SkinThickness	43	+0.4430	+1.5812	−1.1382	1.2956
Insulin	325	+0.5714	+2.1352	−1.5638	2.4456
BMI	39.2	+0.2248	+0.9817	−0.7569	0.5729
DPF	0.528	−0.1265	+0.1695	−0.2960	0.0876
Age	58	+0.8304	+2.1067	−1.2763	1.6289

Sum of $(x-y)^2$ = 3.1750 + 1.9065 + 3.9415 + 1.2956 + 2.4456 + 0.5729 + 0.0876 + 1.6289 = **15.0535**
Distance = √15.0535 = 3.8799

Train #8 (CSV row #451) — Actual: Non-diabetic (0)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	1	+0.6399	−0.8449	+1.4848	2.2046
Glucose	82	−0.7777	−1.3037	+0.5260	0.2767
BloodPressure	64	−1.1901	−0.6938	−0.4963	0.2464
SkinThickness	13	+0.4430	−1.8335	+2.2765	5.1824
Insulin	95	+0.5714	−0.5291	+1.1005	1.2110
BMI	21.2	+0.2248	−1.6381	+1.8629	3.4706
DPF	0.415	−0.1265	−0.1718	+0.0453	0.0020
Age	23	+0.8304	−0.8714	+1.7018	2.8960

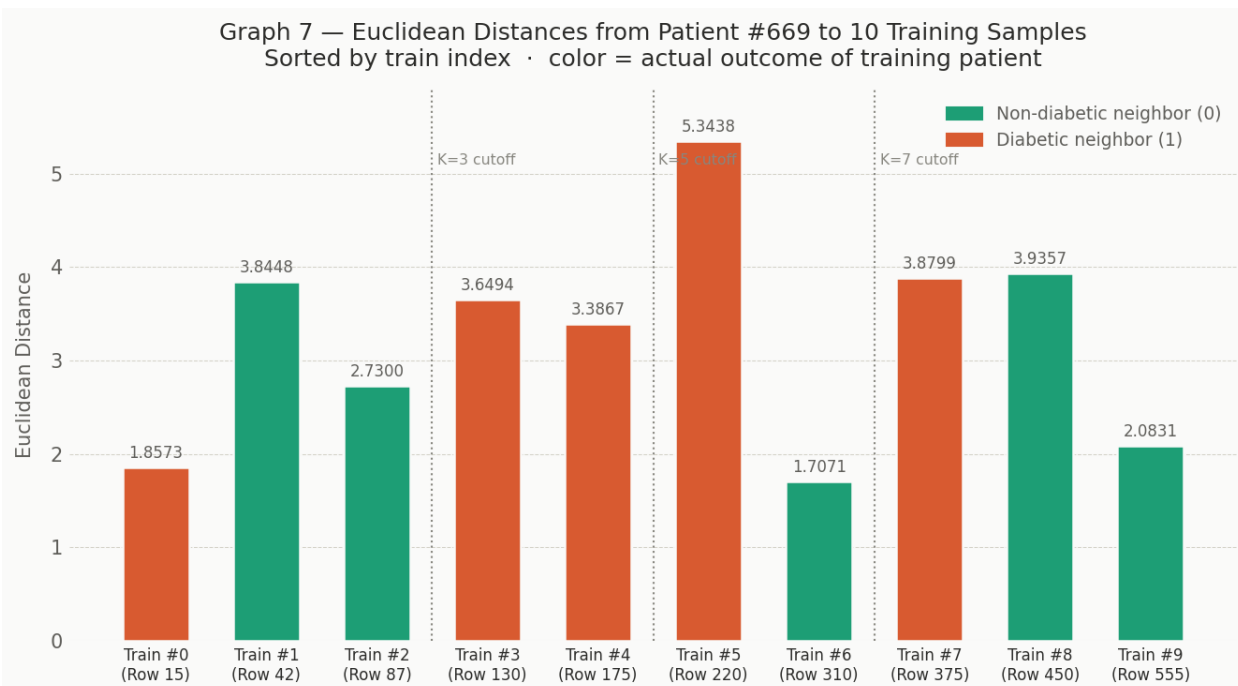
Sum of $(x-y)^2$ = 2.2046 + 0.2767 + 0.2464 + 5.1824 + 1.2110 + 3.4706 + 0.0020 + 2.8960 = **15.4896**
Distance = √15.4896 = 3.9357

Train #9 (CSV row #556) — Actual: Non-diabetic (0)

Feature	Train raw	Test scaled (x)	Train scaled (y)	x − y	$(x-y)^2$
Pregnancies	7	+0.6399	+0.9369	−0.2970	0.0882
Glucose	124	−0.7777	+0.0771	−0.8548	0.7306
BloodPressure	70	−1.1901	−0.1974	−0.9927	0.9854
SkinThickness	33	+0.4430	+0.4430	0.0000	0.0000
Insulin	215	+0.5714	+0.8610	−0.2896	0.0839
BMI	25.5	+0.2248	−1.0123	+1.2371	1.5304
DPF	0.161	−0.1265	−0.9389	+0.8124	0.6600
Age	37	+0.8304	+0.3199	+0.5105	0.2607

Sum of $(x-y)^2$ = 0.0882 + 0.7306 + 0.9854 + 0.0000 + 0.0839 + 1.5304 + 0.6600 + 0.2607 = **4.3391**

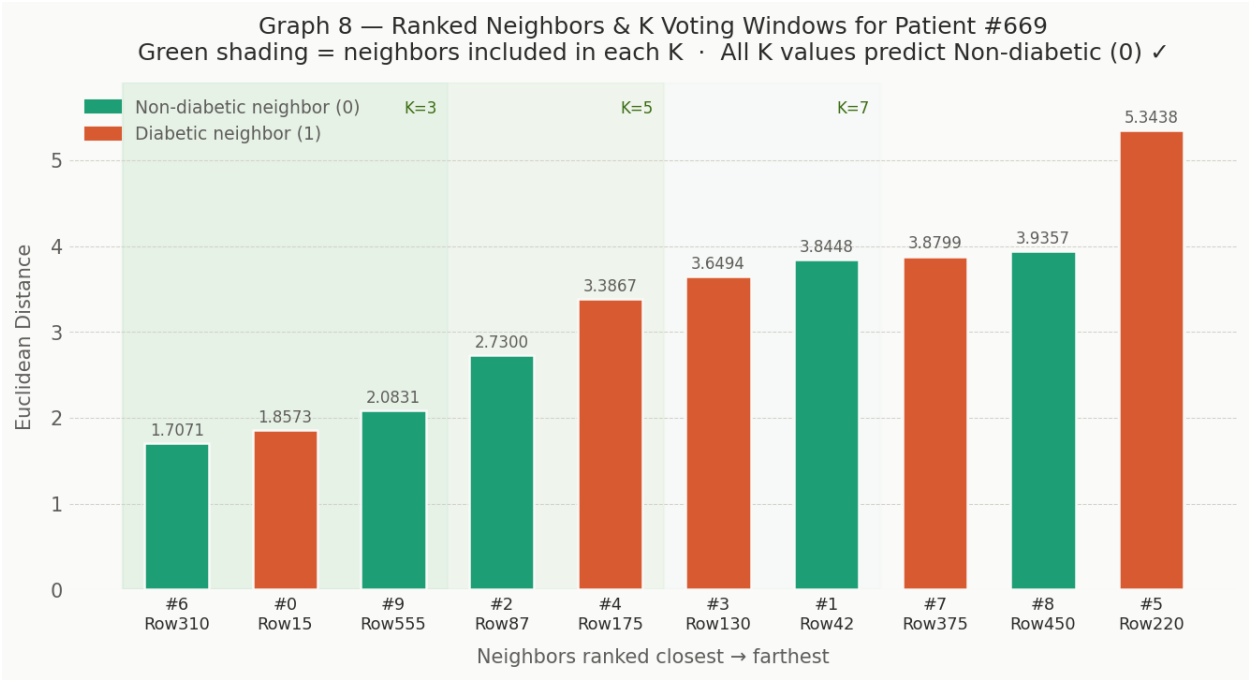
Distance = √4.3391 = 2.0831



Step 1 — Rank All 10 Neighbors by Distance

Rank	Train #	CSV Row	Distance	Outcome
1	Train #6	Row 310	**1.7071**	Non-diabetic (0)

Rank	Train #	CSV Row	Distance	Outcome
2	Train #0	Row 15	**1.8573**	Diabetic (1)
3	Train #9	Row 555	**2.0831**	Non-diabetic (0)
4	Train #2	Row 87	**2.7300**	Non-diabetic (0)
5	Train #4	Row 175	**3.3867**	Diabetic (1)
6	Train #3	Row 130	**3.6494**	Diabetic (1)
7	Train #1	Row 42	**3.8448**	Non-diabetic (0)
8	Train #7	Row 375	**3.8799**	Diabetic (1)
9	Train #8	Row 450	**3.9357**	Non-diabetic (0)
10	Train #5	Row 220	**5.3438**	Diabetic (1)



Step 2 — Majority Vote for K = 3, 5, 7

K = 3 — Top 3 Nearest Neighbors

Rank	Train #	Distance	Vote
1	Train #6 (Row 310)	1.7071	Non-diabetic (0)
2	Train #0 (Row 15)	1.8573	Diabetic (1)
3	Train #9 (Row 555)	2.0831	Non-diabetic (0)

Tally - Non-diabetic: 2 | Diabetic: 1 Predicted Class = Non-diabetic (0)

K = 5 — Top 5 Nearest Neighbors

Rank	Train #	Distance	Vote
1	Train #6 (Row 310)	1.7071	Non-diabetic (0)
2	Train #0 (Row 15)	1.8573	Diabetic (1)
3	Train #9 (Row 555)	2.0831	Non-diabetic (0)
4	Train #2 (Row 87)	2.7300	Non-diabetic (0)
5	Train #4 (Row 175)	3.3867	Diabetic (1)

Tally → Non-diabetic: 3 | Diabetic: 2 Predicted Class = Non-diabetic (0)

K = 7 — Top 7 Nearest Neighbors

Rank	Train #	Distance	Vote
1	Train #6 (Row 310)	1.7071	Non-diabetic (0)
2	Train #0 (Row 15)	1.8573	Diabetic (1)
3	Train #9 (Row 555)	2.0831	Non-diabetic (0)
4	Train #2 (Row 87)	2.7300	Non-diabetic (0)
5	Train #4 (Row 175)	3.3867	Diabetic (1)
6	Train #3 (Row 130)	3.6494	Diabetic (1)
7	Train #1 (Row 42)	3.8448	Non-diabetic (0)

Tally → Non-diabetic: 4 | Diabetic: 3 Predicted Class = Non-diabetic (0)

Final Verdict

K Value	Neighbors Used	Non-diabetic Votes	Diabetic Votes	Predicted	Actual	Correct
K = 3	Rows 310, 15, 555	2	1	Non-diabetic (0)	Non-diabetic (0)	YES
K = 5	Rows 310, 15, 555, 87, 175	3	2	Non-diabetic (0)	Non-diabetic (0)	YES
K = 7	Rows 310, 15, 555, 87, 175, 130, 42	4	3	Non-diabetic (0)	Non-diabetic (0)	YES

Part 4: Model Evaluation (20 pts)

Evaluate your model using:

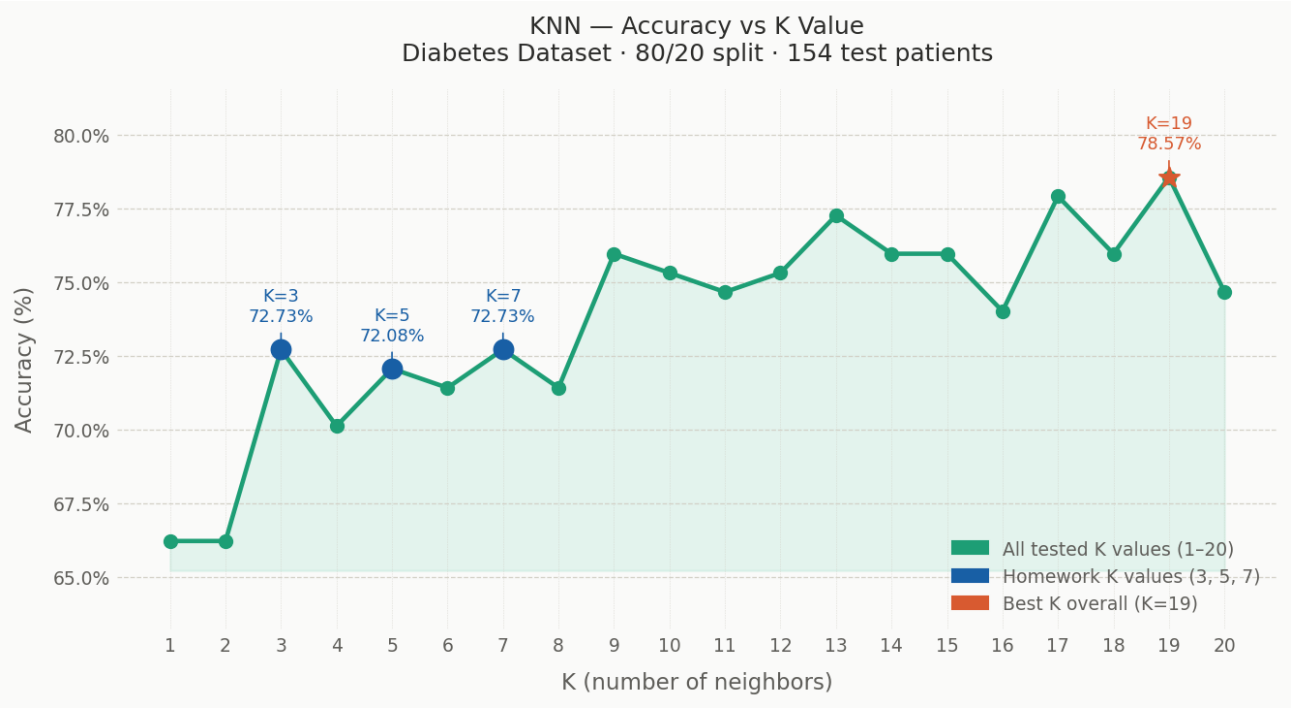
- Accuracy

Accuracy by K value

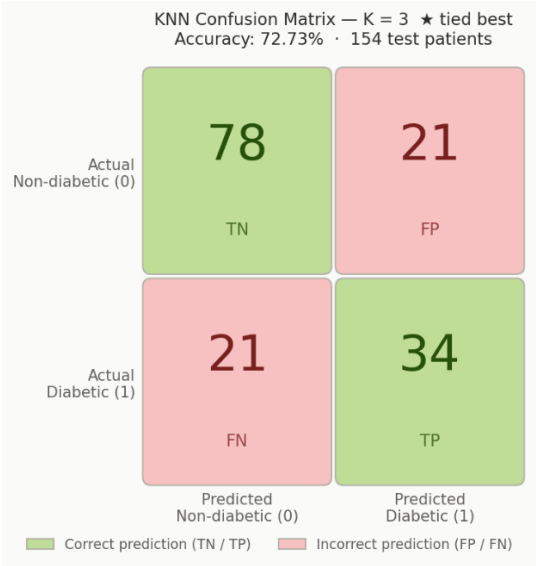
K = 3 == 72.73% (112 correct out of 154 test patients)

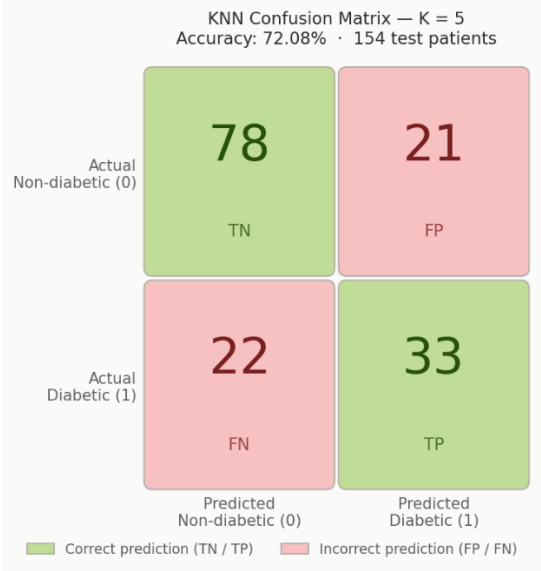
K = 5 == 72.08% (111 correct out of 154 test patients)

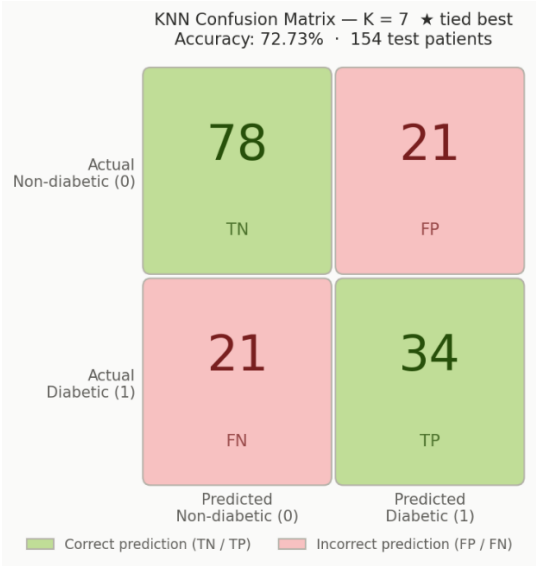
K = 7 == 72.73% (112 correct out of 154 test patients)



- Confusion Matrices


KNN Confusion Matrix — K = 3 ★ tied best
Accuracy: 72.73% · 154 test patients
Actual
Non-diabetic (0)
Actual
Diabetic (1)
78
TN
21
FP
21
FN
34
TP
Predicted
Non-diabetic (0)
Predicted
Diabetic (1)
Correct prediction (TN / TP)
Incorrect prediction (FP / FN)


KNN Confusion Matrix — K = 5
Accuracy: 72.08% · 154 test patients
Actual
Non-diabetic (0)
Actual
Diabetic (1)
78
TN
21
FP
22
FN
33
TP
Predicted
Non-diabetic (0)
Predicted
Diabetic (1)
Correct prediction (TN / TP)
Incorrect prediction (FP / FN)



The test set has 154 patients total:

- 99 are actually Non-diabetic (0)
- 55 are actually Diabetic (1)

The 4 possible outcomes for any prediction

Term	What it means
True Negative (TN)	Patient is actually Non-diabetic, model said Non-diabetic.
True Positive (TP)	Patient is actually Diabetic, model said Diabetic.
False Positive (FP)	Patient is actually Non-diabetic, but model said Diabetic.
False Negative (FN)	Patient is actually Diabetic, but model said Non-diabetic.

Answer the following:

1. Which value of K performed best?

K = 3 and K = 7 are tied at **72.73% accuracy** (112 correct out of 154 test patients), both with identical confusion matrices — 78 TN, 34 TP, 21 FP, and 21 FN. K = 5 came in just below at 72.08%, missing one additional diabetic patient (FN = 22 vs 21). Since K = 3 and K = 7 are equal in accuracy, K = 7 is the better practical choice — it uses more neighbors, making it less sensitive to a single noisy data point, without sacrificing any accuracy here.

2. Why does performance change with different K values?

Each K value draws a different decision boundary around test points. When K is small, the prediction depends on very few neighbors, so the class of whichever training points happen to be closest has outsized influence. When K grows, more neighbors vote, and the majority label starts to reflect the broader neighborhood structure rather than any single point. In this dataset the shift from K = 3 to K = 5 cost one correct diabetic prediction because the 5th nearest neighbor pulled the vote toward non-diabetic for one borderline case. Going from K = 5 to K = 7 added back two neighbors that restored the balance. The accuracy values are close (< 1% apart) because the dataset is moderately balanced and the decision boundary between diabetic and non-diabetic is not dramatically sharp.

3. What happens when K is too small or too large?

When K is **too small** (e.g., K = 1), the model essentially memorizes the training data. One mislabeled or noisy neighbor can flip the entire prediction. This leads to high variance — the model is very sensitive to local quirks and performs poorly on unseen data (overfitting). When K is **too large** (e.g., K = 200 out of 614 training samples), the model starts consulting neighbors that are far away and genuinely dissimilar to the test point. The majority vote converges toward the dominant class in the training set (non-diabetic, at 65%) regardless of the actual patient's features. This leads to high bias — the model loses its local sensitivity and underfits, systematically misclassifying the minority class (diabetic patients). The sweet spot is a moderate K that is large enough to smooth out noise but small enough to stay local and meaningful.

Part 5: Analysis and Reflection (20 pts)

Provide a discussion (minimum 300 words):

Strengths of KNN

K-Nearest Neighbors is one of the most intuitive algorithms in machine learning, and its simplicity is arguably its greatest strength. There is no training phase in the traditional sense — the model does not learn parameters or build an internal representation of the data. Instead, it memorizes the entire training set and defers all computation to prediction time. This means the model is always up to date: adding new training data requires no retraining whatsoever, just appending to the existing dataset. For a medical dataset like this one, that property is genuinely useful because patient records accumulate continuously.

KNN is also non-parametric, meaning it makes no assumptions about the underlying distribution of the data. Many algorithms, such as logistic regression, assume that the relationship between features and outcome follows a specific mathematical form. KNN does not. It simply looks at what is nearby in feature space and lets the neighbors vote. This makes it naturally capable of capturing complex, non-linear decision boundaries that parametric models would struggle to represent. If diabetic and non-diabetic patients overlap in subtle, irregular ways across the eight features, KNN can adapt to that without any architectural change — just by adjusting K.

Another strength is its transparency. When KNN makes a prediction, you can directly inspect which training patients influenced it and examine their feature values. This is meaningful in a healthcare context where a clinician might want to understand why a patient was flagged as diabetic. Rather

than a black-box score, KNN can point to ten real historical patients who were most similar to the new one — a form of explainability that many modern algorithms cannot offer.

Limitations of KNN

Despite its elegance, KNN carries significant practical drawbacks. The most immediate is computational cost at prediction time. Because no model is built during training, every single prediction requires computing the distance from the test instance to all 614 training patients across all 8 features. With 154 test patients in this experiment, that means over 94,000 individual distance calculations. At the scale of this dataset that is manageable, but in production systems with hundreds of thousands of records and real-time prediction requirements, this becomes a serious bottleneck. The algorithm does not scale gracefully.

KNN is also extremely sensitive to feature scaling, which is precisely why standardization was a required preprocessing step in this homework. Consider Insulin values, which ranged from 14 to 846 mU/L in the raw data, versus DiabetesPedigreeFunction values ranging from 0.078 to 2.42. Without scaling, the Euclidean distance computation would be dominated almost entirely by Insulin simply because its numerical range is so much larger. A patient's DPF — a clinically meaningful hereditary risk factor — would contribute almost nothing to the distance calculation. After standardization, every feature contributes equally, giving each one a fair influence on the neighborhood. This sensitivity means that any new data point at prediction time must also be scaled using the same mean and standard deviation computed from the training set, adding a layer of preprocessing dependency that must be carefully managed.

A third limitation is sensitivity to irrelevant features. KNN treats all features as equally meaningful by default. If the dataset contained noisy or redundant columns, they would pollute the distance calculation and cause the model to identify neighbors that appear close mathematically but are not actually clinically similar. Feature selection becomes critically important, and KNN provides no built-in mechanism for identifying or down-weighting irrelevant dimensions.

Finally, KNN is sensitive to class imbalance and to outliers. In this dataset, 500 out of 768 patients (65%) are non-diabetic, which creates an inherent bias toward predicting the majority class, especially as K grows larger and the neighborhood begins to reflect the overall class distribution rather than local structure.

When KNN Is Appropriate or Not Appropriate

KNN is well suited to situations where the dataset is small to medium in size, the relationships between features and outcome are non-linear and complex, and interpretability of individual predictions matters. It performs best when all features are genuinely informative, the data is properly scaled, and the classes are reasonably balanced. It is also a strong baseline algorithm — quick to implement and useful for benchmarking before committing to more complex approaches.

KNN is not appropriate when prediction speed is critical, since there is no precomputed model to query. It is equally ill-suited to very high-dimensional data, where the notion of distance becomes

unreliable — a phenomenon known as the curse of dimensionality, where all points in high-dimensional space tend to become equidistant from one another, stripping the nearest-neighbor concept of its meaning. It is also a poor choice when storage is limited, since the entire training set must be retained in memory indefinitely, and when the dataset contains many irrelevant or redundant features that would corrupt the distance metric.

Observations from the Experiment

Several concrete patterns emerged from running KNN on the diabetes dataset. The most striking was the accuracy curve across K values from 1 to 20. At K=1 and K=2, accuracy sat at just 66.23% — the model was overfitting, allowing individual noisy neighbors to dominate predictions. As K increased through the homework range of 3, 5, and 7, accuracy climbed into the 72% range, stabilizing as the majority vote smoothed out local noise. Accuracy continued rising beyond the homework values, peaking at K=19 with 78.57%, suggesting that the boundary between diabetic and non-diabetic patients in this dataset is relatively smooth rather than sharply jagged, and that larger neighborhoods capture it more faithfully.

The confusion matrices revealed a consistent and clinically important pattern across all three K values: the model was significantly better at identifying non-diabetic patients (TN rate of 78/99, or 78.8%) than diabetic ones (TP rate of 34/55, or 61.8% for K=3 and K=7). This asymmetry is directly tied to class imbalance. With 65% of the training data being non-diabetic, the neighborhood around any test patient tends to have more non-diabetic representatives simply by proportion, biasing predictions toward zero. In a real clinical deployment, this pattern would be dangerous — false negatives, where diabetic patients are told they are healthy, carry far greater consequences than false positives. This observation strongly suggests that for medical applications, KNN should be tuned not just for overall accuracy but specifically to minimize false negatives, potentially by using a weighted voting scheme or by adjusting the classification threshold.

The imputation step also proved important. Columns like Insulin and SkinThickness had 374 and 227 zero values respectively — biologically impossible readings that represented missing data. Leaving these in would have caused patients with missing measurements to appear artificially similar to one another in distance calculations, clustering them together for the wrong reasons entirely. Replacing zeros with column medians computed from valid values preserved the distributional integrity of each feature and ensured that distance calculations reflected genuine physiological similarity rather than shared data absence.

IMPORTANT NOTES

• You must show your own computations for the manual KNN step

• Clearly explain your methodology

• Avoid copying code without understanding

• Late submissions may be penalized

BONUS (Optional: +10 pts)

• Visualize results using graphs (e.g., accuracy vs K)

• Compare KNN with another algorithm (e.g., Logistic Regression)

SUBMISSION REQUIREMENTS

Submit the following:

1. PDF Report containing:

o Answers to all parts

o Tables, computations, and explanations

o Graphs

2. Code File (upload to github.com):

o Python (.py) or Jupyter Notebook (.ipynb)